

Transitions: DS + Algorithms

Binary Search
Trees



Worksheet:

#1

pre:

in:

post: ✓

root

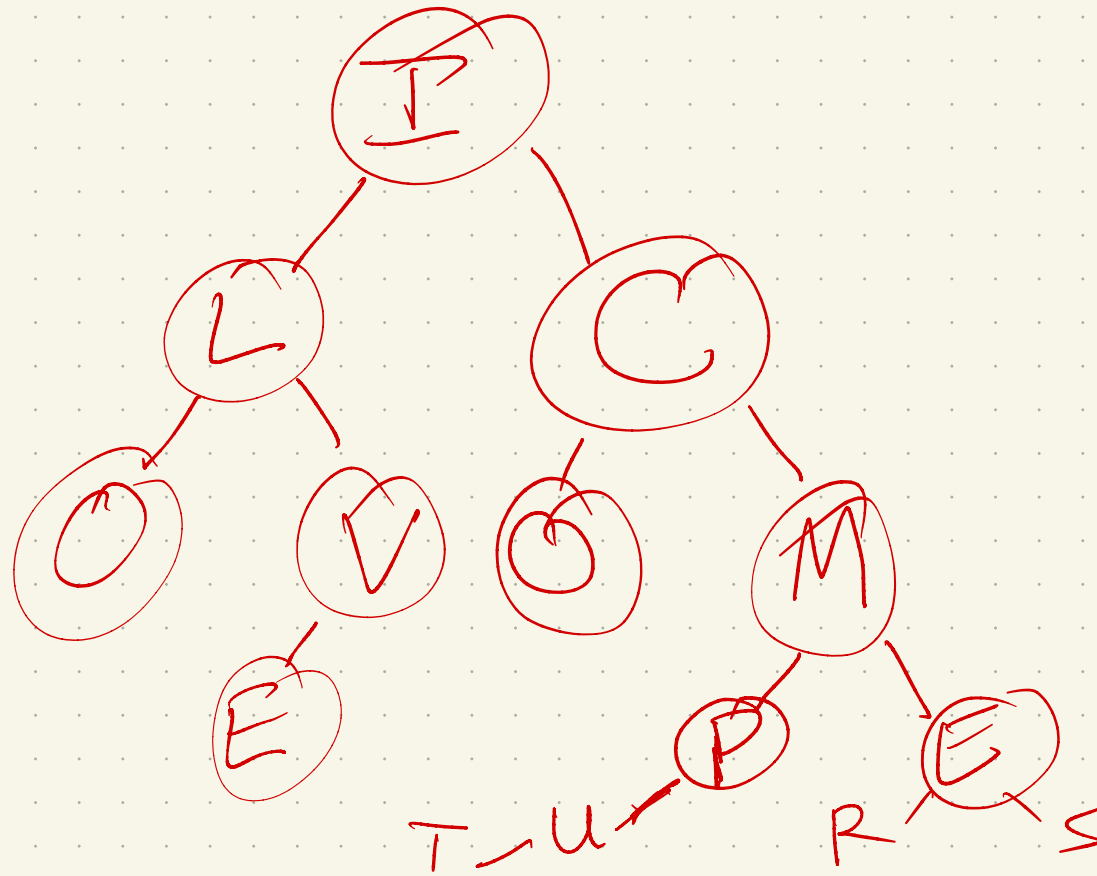
#2

2. Consider the following two tree traversal outputs:

Preorder: I LOVE COMPUTERS

Inorder: O L E V I O C T U P M R E S

Draw the binary tree which results in these two outputs for the specified traversals.



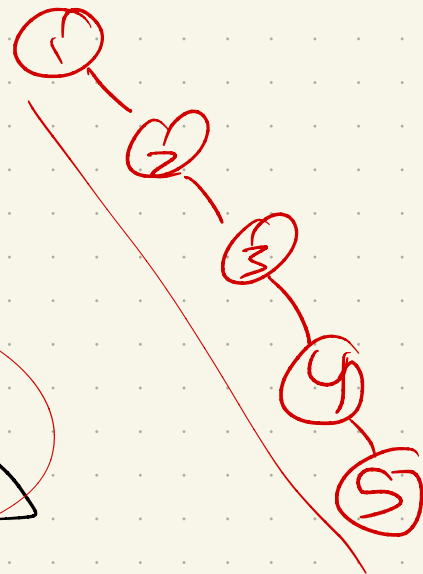
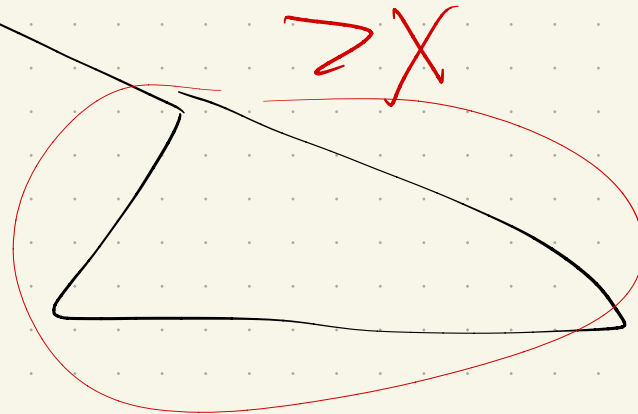
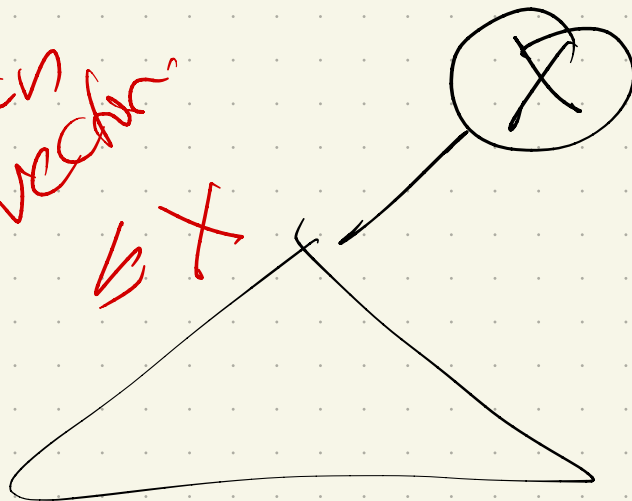
Last time: Binary Search Trees

Definition:

A binary tree where we maintain the following:

- The value at any node is $\geq$ its left child, and $<$ its right child

Goal:
Better than
a list or vector
 $\leq X$



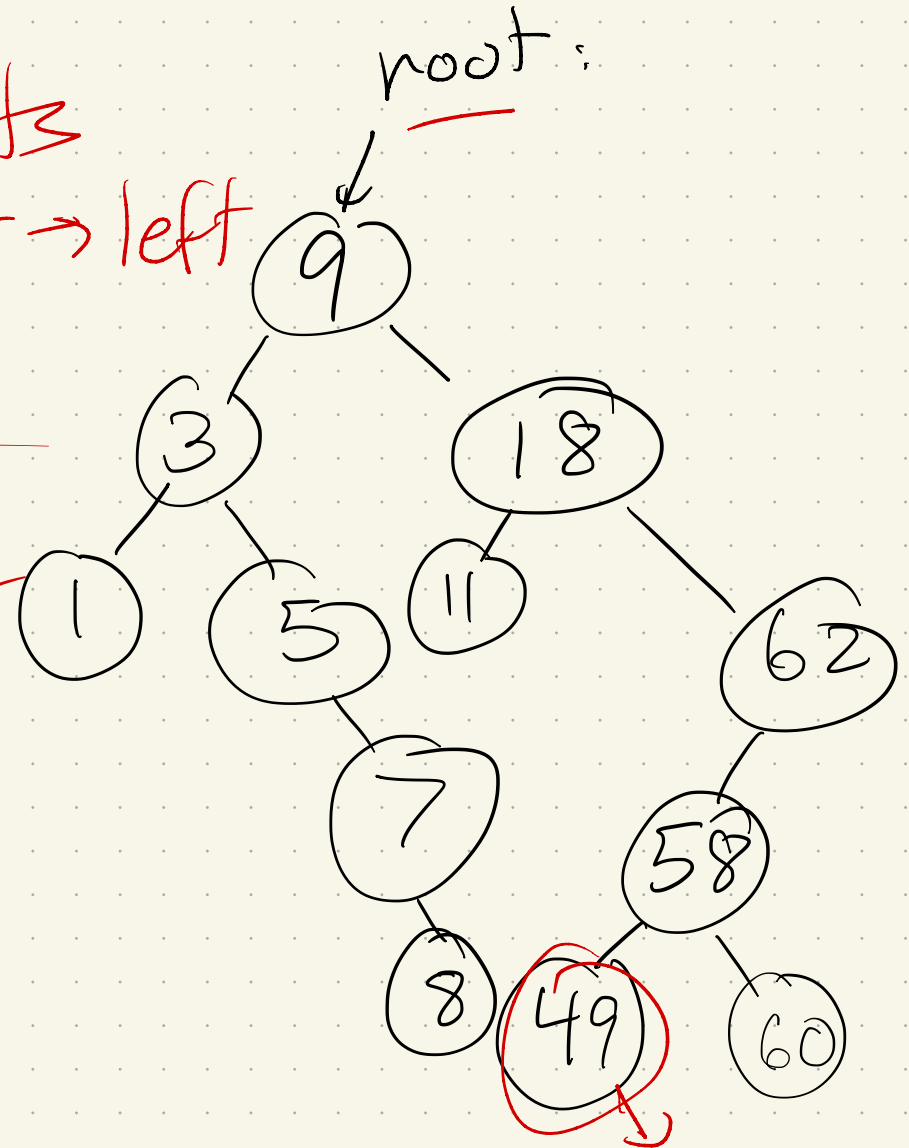
Finding in a BST: $\leftarrow O(\text{height of tree})$
 $= O(n)$

- Check if root = target value
return true

- If root > target
if root $\rightarrow$ left exists
call find on root $\rightarrow$ left
else return false

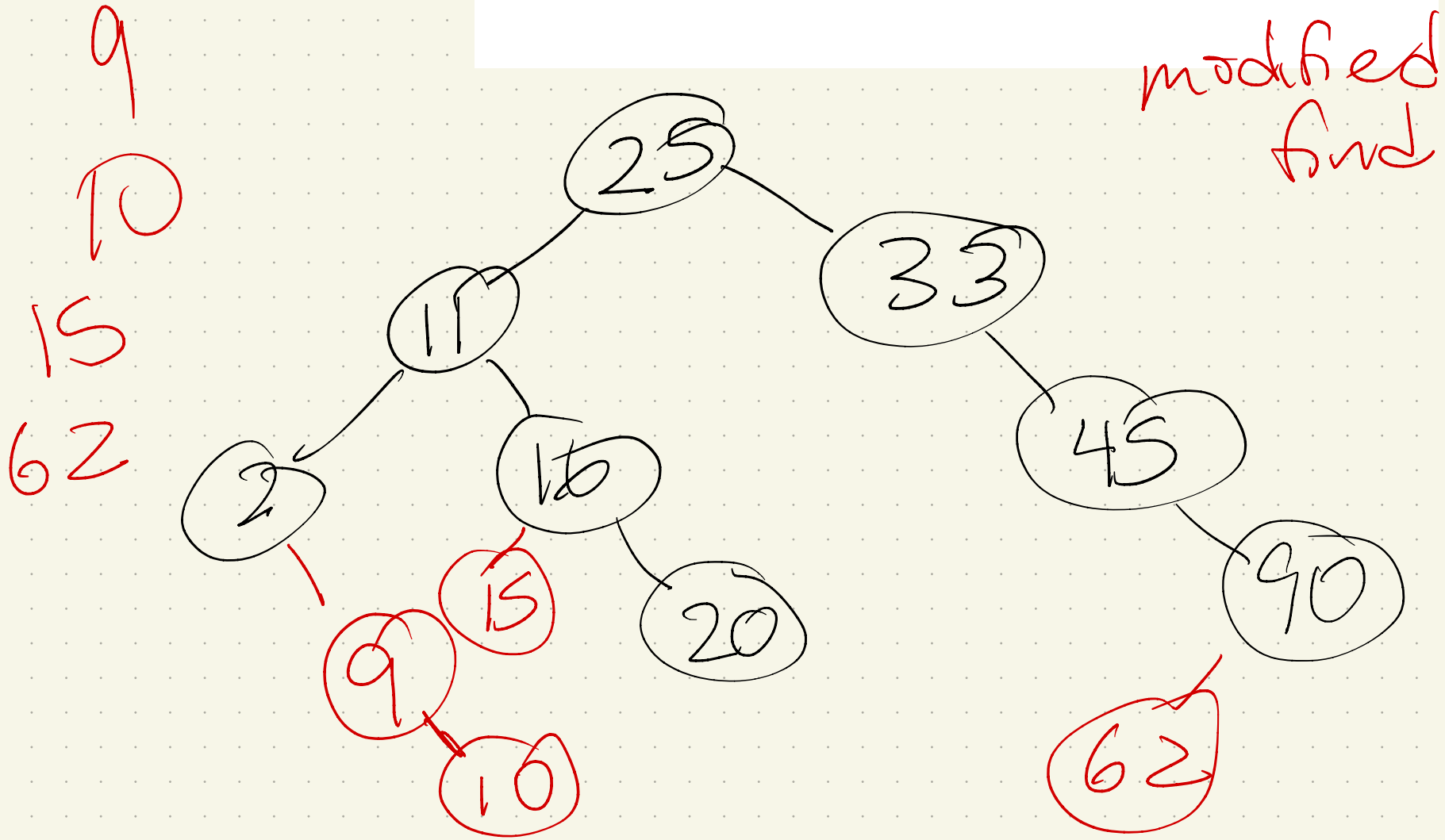
- If root < target
if $\rightarrow$ call find on root $\rightarrow$ right
else false

Ex:
find(58)
find(50)



Insert:

```
1 function insert(value, node)
2   if value equals node.Key
3     # we have a duplicate key
4     return FOUND
5   else if value < node.Key
6     if node has left child
7       insert(value, node.LeftChild)
8   else
9     set node.LeftChild to new node with Key as value
10  else if value > node.Key
11    if node has right child
12      insert(value, node.RightChild)
13  else
14    set node.RightChild to new node with Key as value
```



Runtime:

Run find $\hookrightarrow$ $O(\text{height of tree})$

$\hookrightarrow$ + insert new node
as a leaf

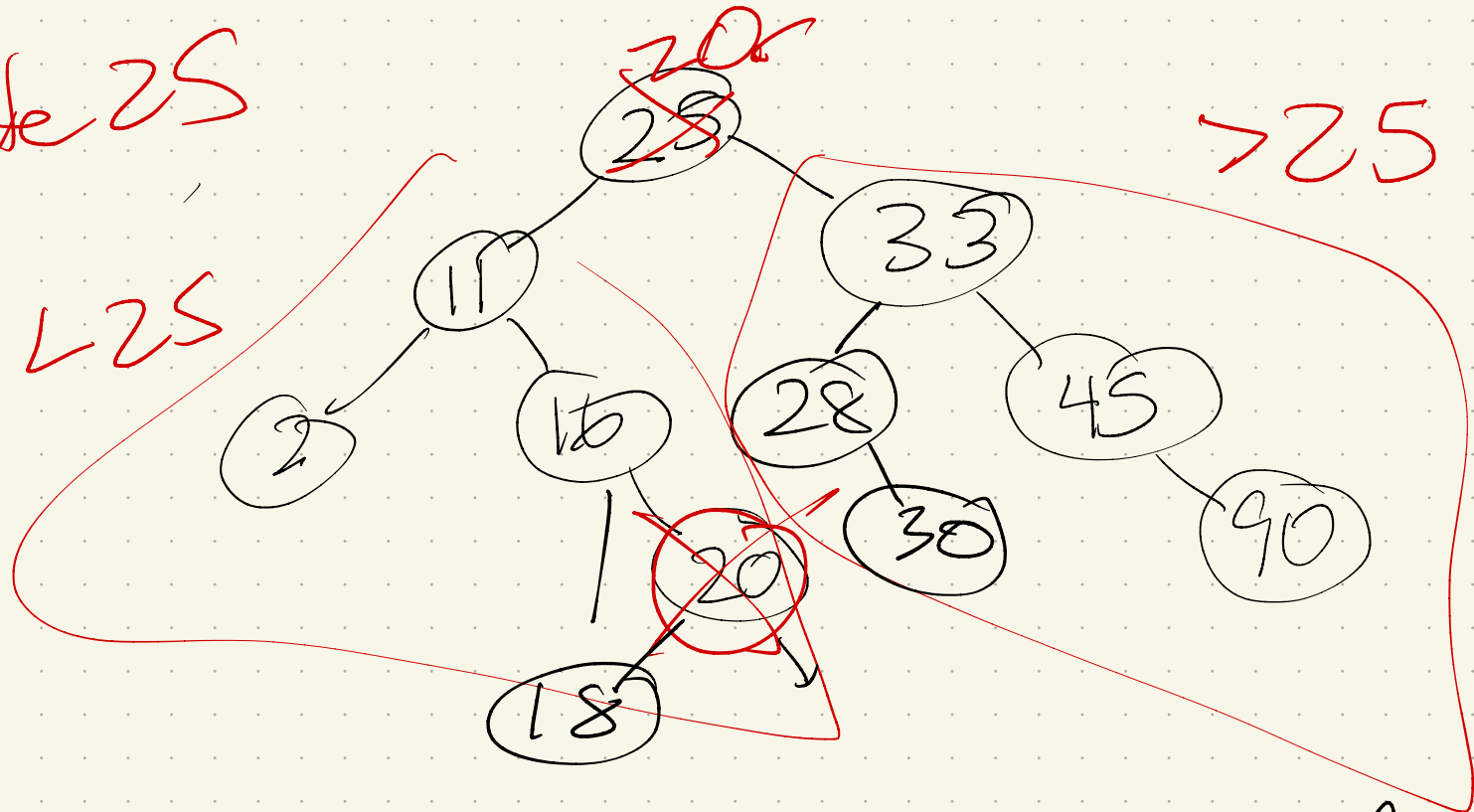
$O(1)$: create node
& update ptrs

Note: Delete is a bit more complex!

delete 25

L 25

> 25



Some details, but can always find

some node to move while

keeping it a BST.

↳ (height of tree)

Balanced BSTs

Many kinds:

- Red-black trees

- Splay trees

- AVL trees

⋮

← stronger: $\leq 1.56 \lg n$
?

Compare
vector or List

Goal of them all:

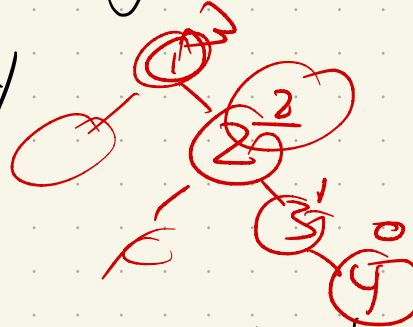
Find, Insert,
& delete $O(\lg n)$

How? Make sure height $\leq c \cdot \lg n$

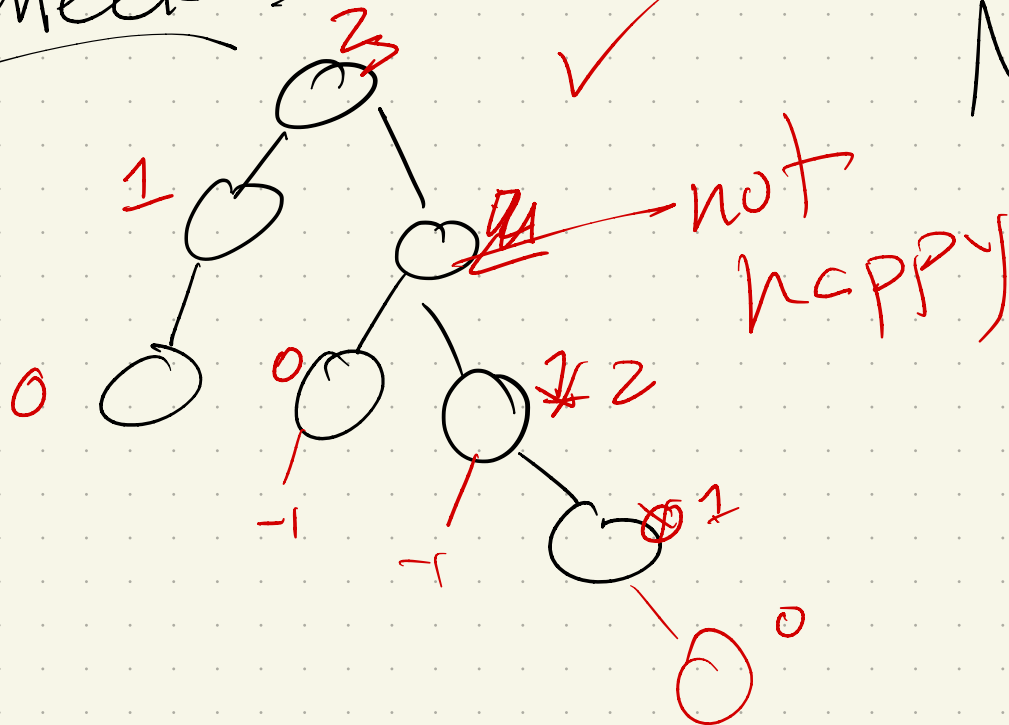
AVL trees: any BST with this prop

Height balance property:

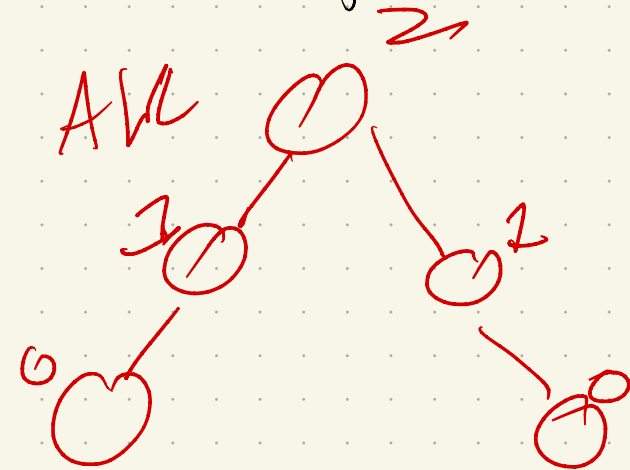
For every node v in T , the heights of v 's children differ by at most 1.

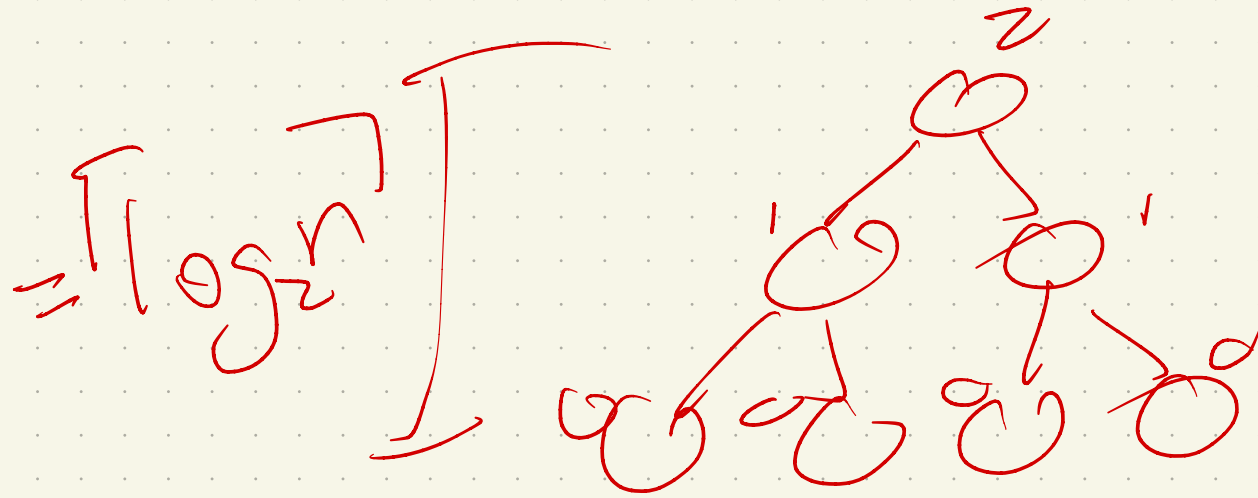


Check:

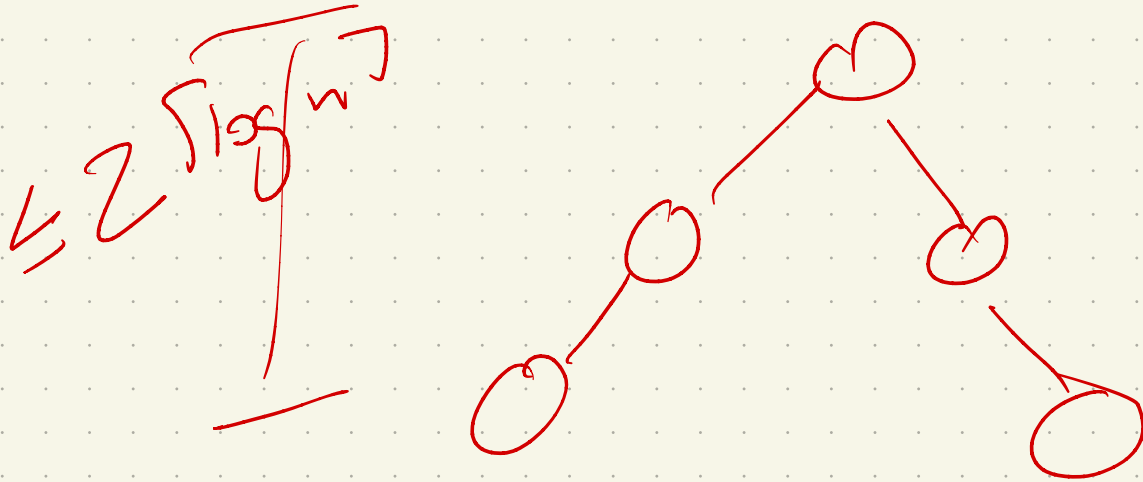


Note: not complete!





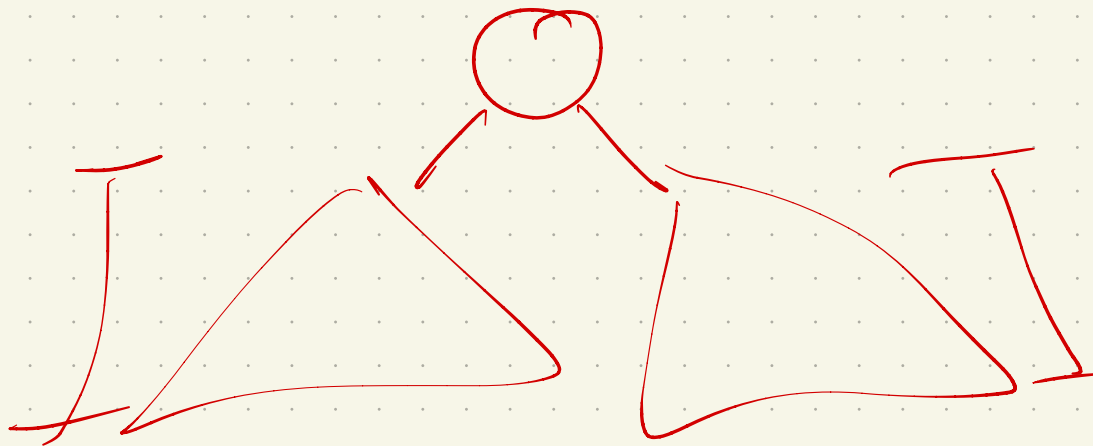
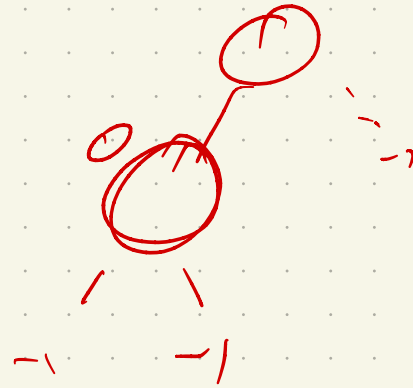
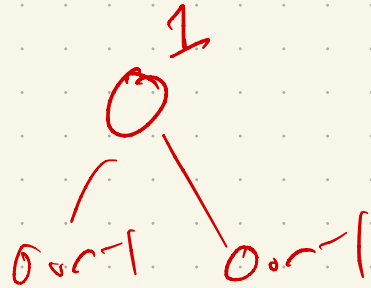
AVL
tree



Claim:

The height of an AVL tree
with n nodes is $\leq 2 \lceil \log_2 n \rceil$?

induction! on height of tree:



Then: how to maintain?

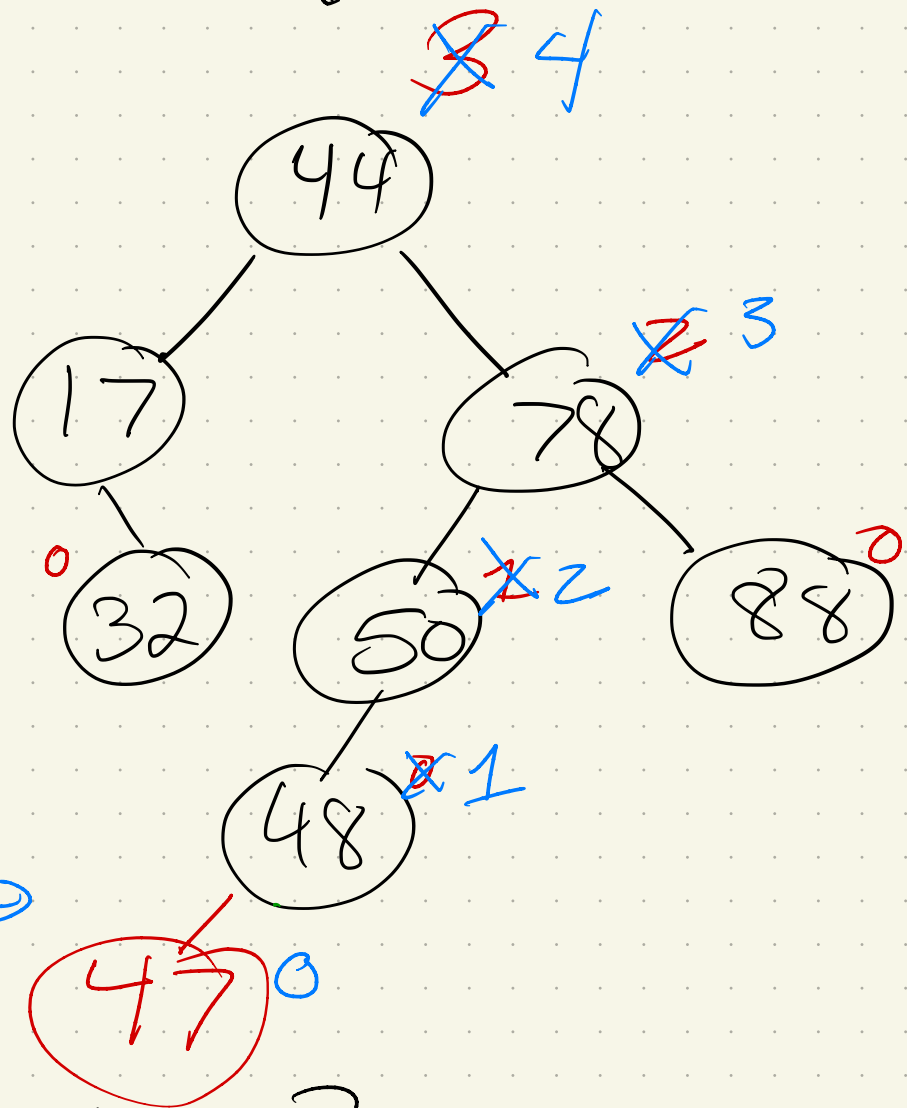
insert(2)

insert(47):

Still a

BST!

after insert
heights



Where is the violation?

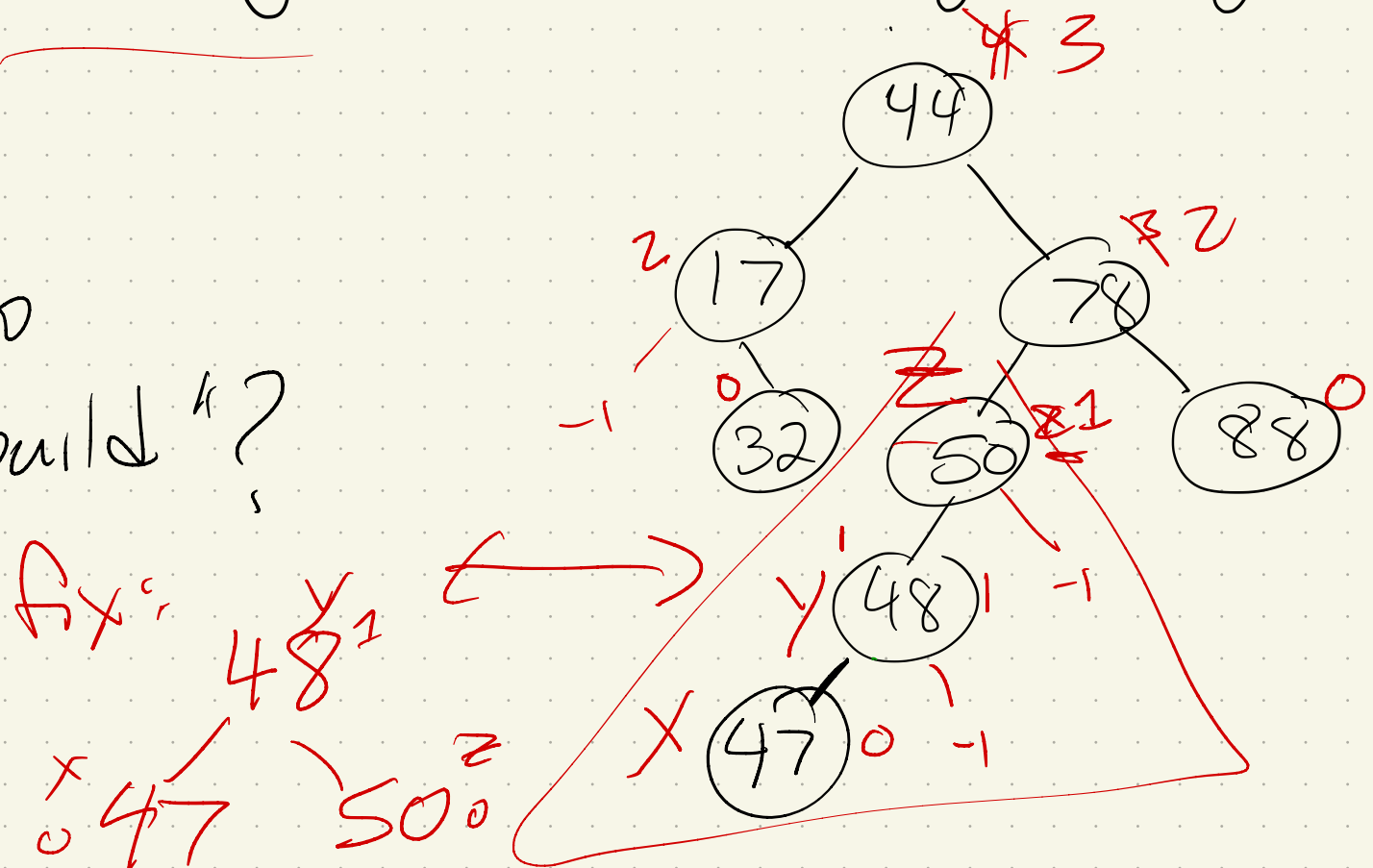
path from new node to root
might violate HB prop

Consider the lowest node which fails the Height-Balance property.

↳ call this z

- Let y be child w/ bigger height.
- Let x be grandchild w/ larger height

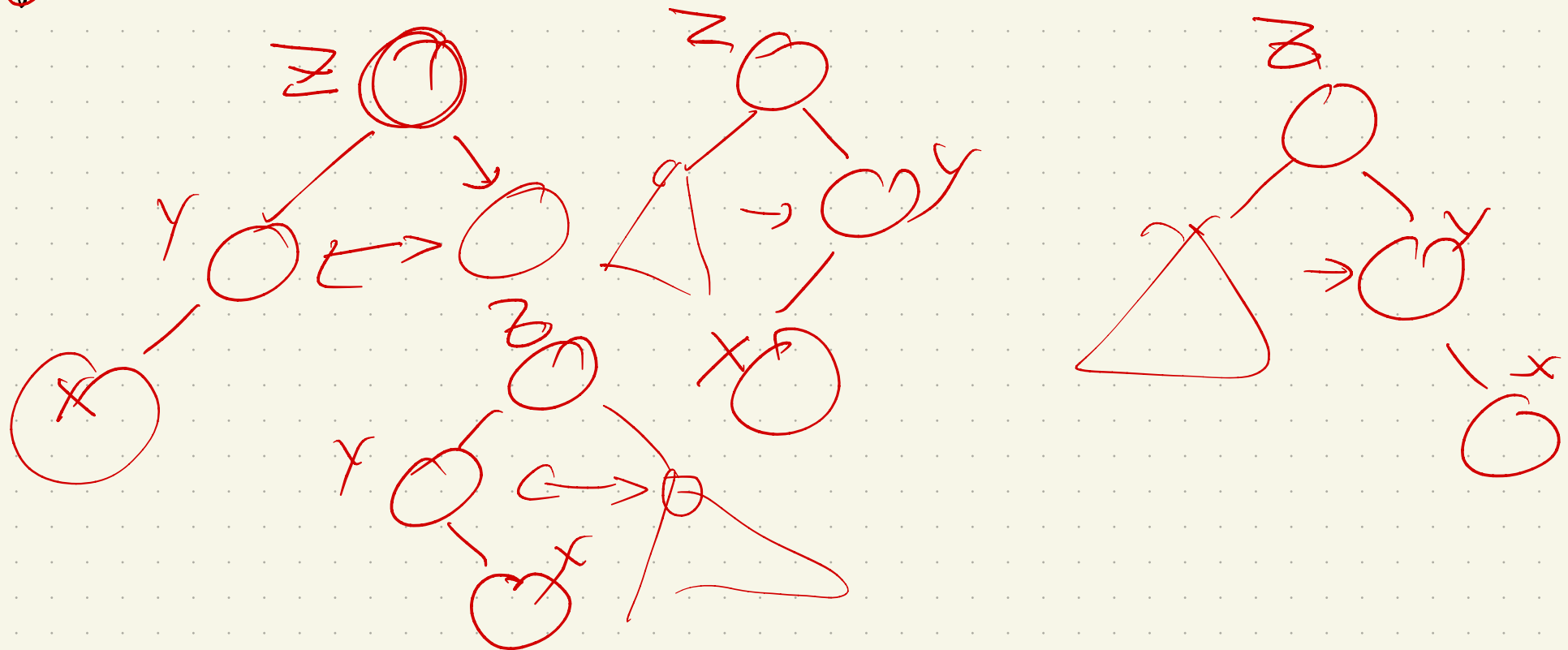
How to
"rebuild"?

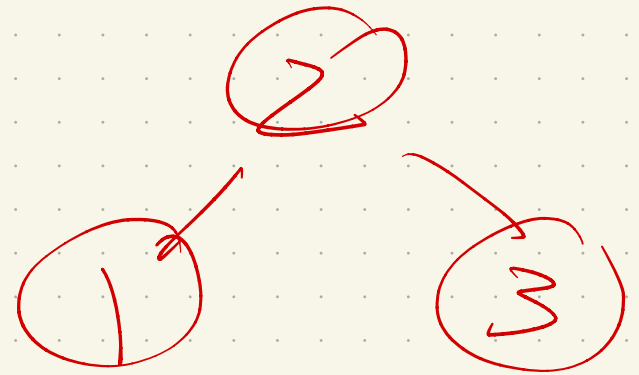
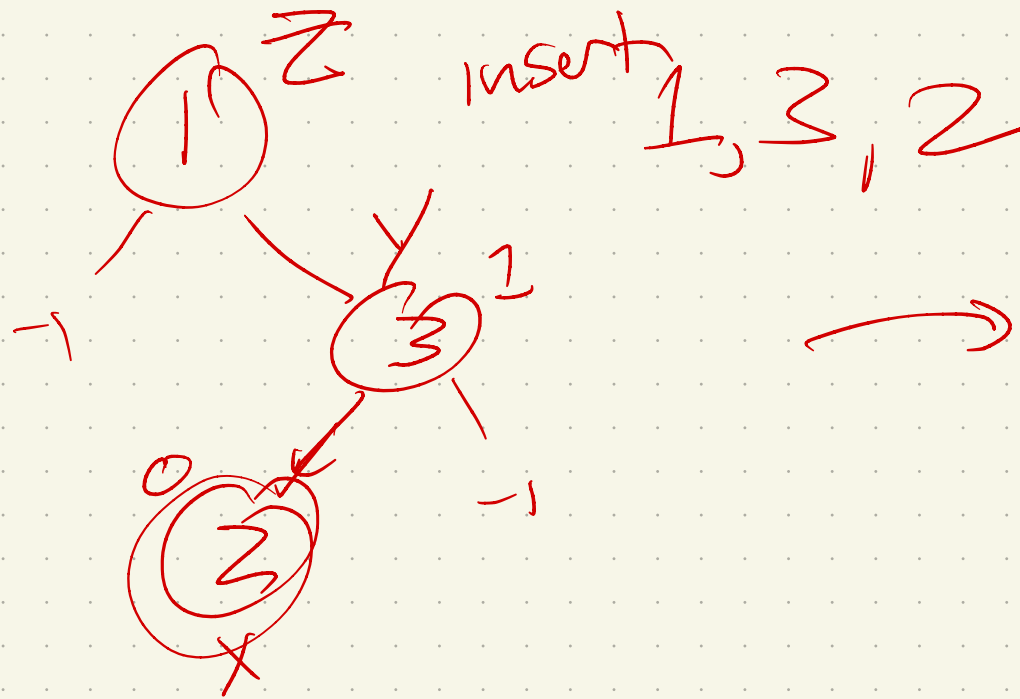
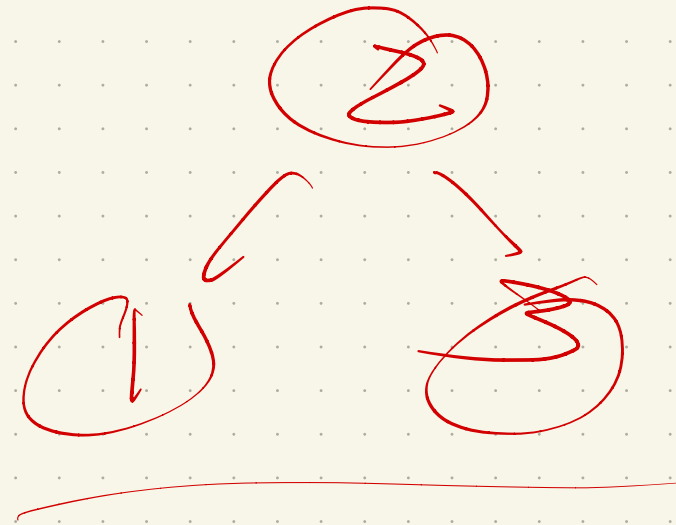
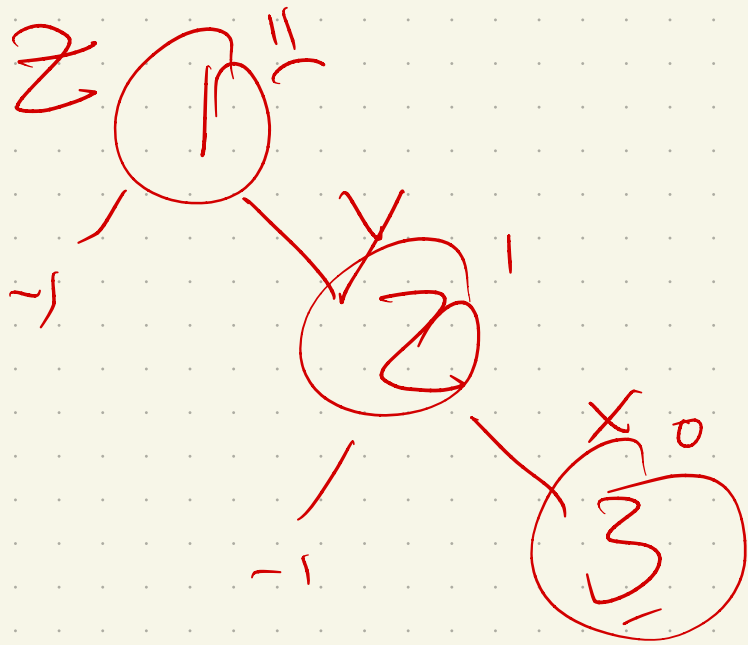


Extrapolate:

$X \rightarrow Y \rightarrow Z$
How could they order?

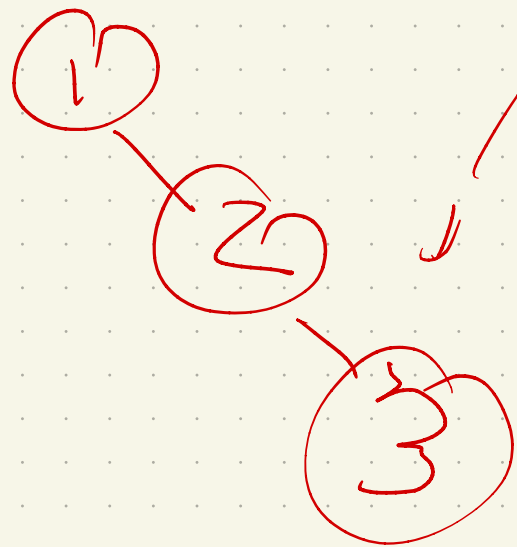
~~Rebalance:~~



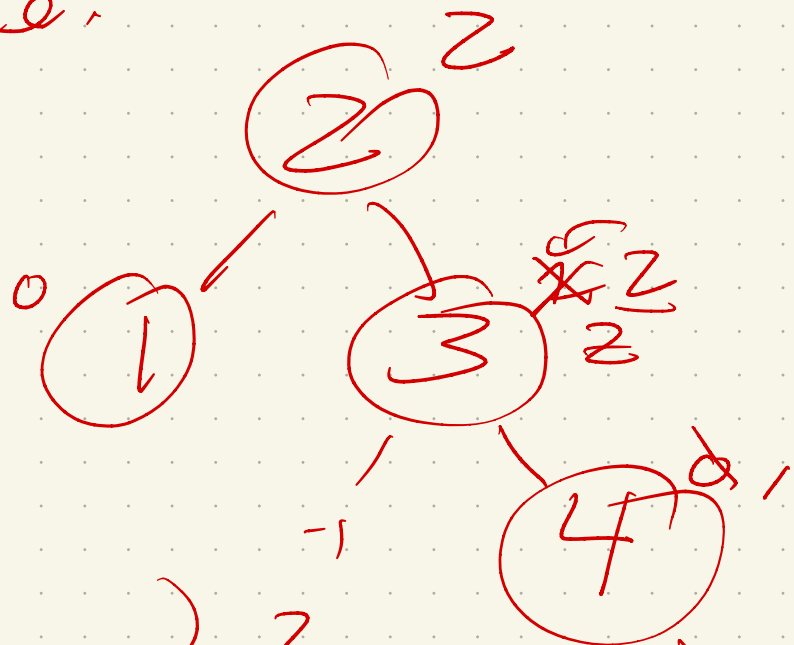


Bigger example:

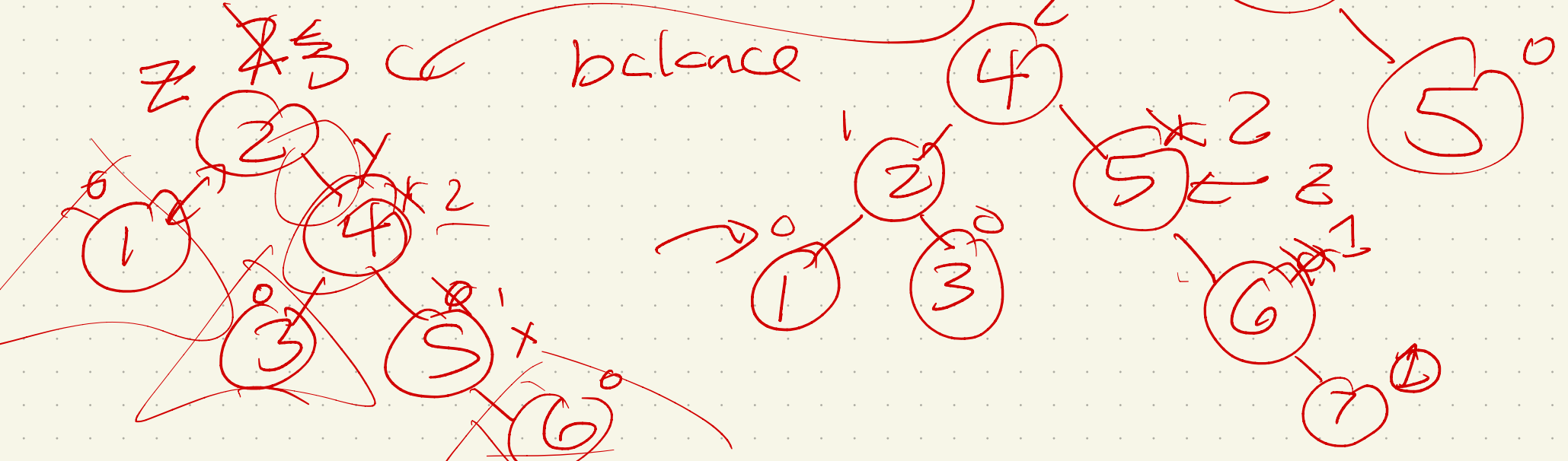
Insert: 1, 2, 3, 4, 5, 6, 7

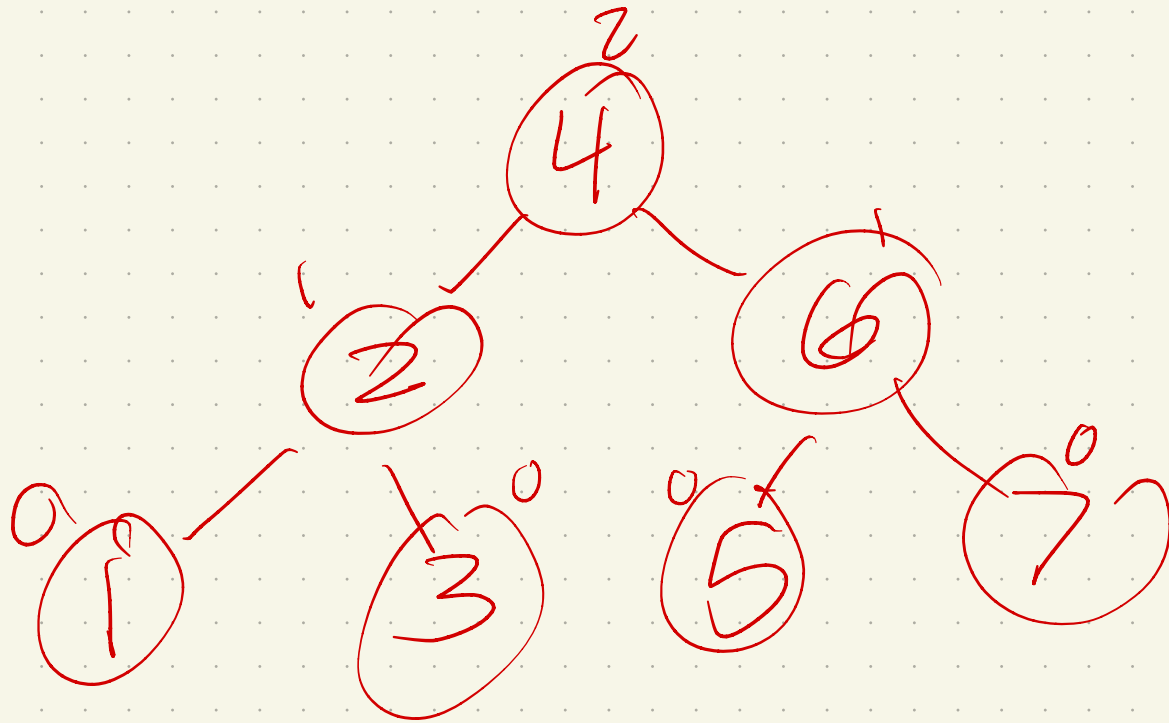


rebalance:



balance





→ $\lceil \log_2 7 \rceil = 3$

Inserting 1 value: $O(\log n)$

Runtime:

Insert as BST: $O(\text{height})$
 $= O(\log n)$

→ need to fix:
traverse back up from
new node to root, calculating
height: $O(\log_2 n)$

→ $O(\log n)$